TwoPhaseAMR.jl

Technical reference

Method outline and implementation map

Based on the notation of Remmerswaal [20]

Contents

1 Method overview **1**
- 1.1 Models and scope . . . 1
- 1.2 Time-step stages . . . 1
- 1.3 Accuracy and model limits . . . 2

2 Block-tree AMR mesh **3**
- 2.1 Blocks, levels, and grading . . . 3
- 2.2 Ownership and schedules . . . 3
- 2.3 Boundaries and halos . . . 3

3 Discrete setting on AMR **5**
- 3.1 Control volumes and functions . . . 5
- 3.2 Hanging faces and dual volumes . . . 5
- 3.3 Compatible operators . . . 5

4 Interface description and curvature **7**
- 4.1 Fractions and reconstruction . . . 7
- 4.2 Interface initialisation . . . 7
- 4.3 Height functions and contact angle . . . 7

5 Geometric VOF transport **9**
- 5.1 Directional WY sweeps . . . 9
- 5.2 Fraction bounds and CFL . . . 9
- 5.3 Two-velocity interface carriers . . . 10

6 Momentum transport **11**
- 6.1 Staggered fluxes and dual geometry . . . 11
- 6.2 Flux interpolants and limiters . . . 11
- 6.3 Phase momentum transport . . . 12

7 Cut cells and embedded solids **13**
- 7.1 Open geometry and PLIC clipping . . . 13
- 7.2 Cut-cell operators on AMR . . . 13

8 Surface tension and gravity **15**

8.1 CSF and conservative ghost-fluid forces . . . 15
8.2 Gravity and well-balancedness . . . 16

9 Viscous terms **17**
9.1 Single-phase viscous step . . . 17
9.2 Two-phase viscosity and equivalent velocity . . . 17

10 Pressure projection **19**
10.1 One-velocity projection . . . 19
10.2 CMDGFM jump interpolation . . . 19
10.3 Mimetic projection . . . 20
10.4 Velocity and pressure boundaries . . . 20

11 Operators and linear solvers **21**
11.1 Gradient and elliptic operators . . . 21
11.2 Tree geometric multigrid . . . 21
11.3 Krylov methods and tolerances . . . 21

12 Refinement criteria and conservative transfer **23**
12.1 Refinement criteria . . . 23
12.2 Cell, face, and phase transfer . . . 23
12.3 Divergence preservation and regridding . . . 24

13 Stability and accuracy **25**
13.1 Advective and capillary time steps . . . 25
13.2 Gravity time-step limit . . . 25
13.3 Splitting and spatial accuracy . . . 25

14 Verification **27**
14.1 Convergence and reference cases . . . 27
14.2 Conservation and balance . . . 27

A Deviations from the thesis **29**

B Notation **31**

C API index **33**

Bibliography **35**

Chapter 1

Method overview

This chapter will map the incompressible one- and two-velocity models and a complete time step. Thesis Chapters 2 and 7 describe the governing equations and two-phase step ordering [20]; the order of the complete single-phase discretisation requires a separate analysis.

1.1 Models and scope

The implemented geometries are two-dimensional; the physical models are incompressible.

Deviation 1.1: Dimensionality and compressibility

The thesis also treats three dimensions and compressible extensions; neither is implemented here.

Implementation

`SinglePhase`, `TwoPhase`, `OneVelocityModel`, `TwoVelocityModel`.

1.2 Time-step stages

The thesis notation uses u^* after transport, u^{**} after diffusion, and u^{***} after forces, followed by projection. Body forces and boundary updates are also part of the implementation sequence.

Implementation

`FlowSimulation`, `step!`, `simulate!`.

1.3 Accuracy and model limits

The formal order depends on the model and splitting; no global second-order claim is made in this outline. See Chapter 13.

Implementation

`NavierStokesOptions`, `stable_timestep`.

Chapter 2

Block-tree AMR mesh

This chapter will describe tree topology and schedules for a graded quadtree. Thesis Chapter 2 introduces graded refinement and the mesh-width hierarchy, but does not describe this package's block-tree implementation [20].

2.1 Blocks, levels, and grading

Let $\mathcal{T}$ be the block tree, $\mathcal{L}$ its leaf set, $\mathcal{B}$ a block, and L the maximum refinement level. The spacing at level ℓ is $2^{-\ell}h_0$.

Implementation

`BlockTreeMesh`, `BlockKey`, `LeafBlock`.

2.2 Ownership and schedules

The leaf-cell and face schedules, including hanging segments $\mathcal{H}(f)$, are the units of mesh traversal.

Implementation

`CellRef`, `FaceRef`, `isbalanced`.

2.3 Boundaries and halos

This section will describe periodic wrap, halo exchange, and same- and mixed-level neighbours.

Implementation

`fill_halo!`, `VelocityBoundaryConditions`.

Chapter 3

Discrete setting on AMR

Thesis Chapters 2, 6, and 7 introduce control-volume sets, function spaces, and compatible operators [20]. This chapter will extend that setting to hanging faces, dual volumes, and vertex patches on the block tree.

3.1 Control volumes and functions

The sets $\mathcal{C}$ and $\mathcal{F}$ denote leaf cells and their faces; $\vec{\mathcal{C}}$ and $\vec{\mathcal{F}}$ are the respective function spaces.

Implementation

`CellField`, `FaceField`, `FaceVelocity`.

3.2 Hanging faces and dual volumes

The incidence fraction $\alpha_{c,f}$ describes the part of cell c associated with face f; ω_f denotes the staggered control volume.

Implementation

`BlockTreeMesh`, `StaggeredVelocityAdvectionWorkspace`.

3.3 Compatible operators

The discrete divergence $\mathfrak{D}$ and gradient $\mathfrak{S}$ and their AMR adjointness and interpolation factorisation will be specified here.

Implementation

`GradientOperator`, `divergence!`, `gradient!`.

Chapter 4

Interface description and curvature

Thesis Chapters 2–5 cover the interface, reconstruction, and curvature including local and geometric height functions [20]. This chapter will map the implemented PLIC representation, initialisation, curvature fallbacks, and contact angles.

4.1 Fractions and reconstruction

The liquid and gas fractions are denoted α^{l} and α^{g}.

Deviation 4.1: Reconstruction family

Optimisation-based MOF/LVIRA and parabolic PLVIRA/PROST/PMOF reconstructions from the thesis are not implemented.

Implementation

`InterfaceField`, `reconstruct!`.

4.2 Interface initialisation

Level-set quadrature and exact polygon clipping initialise the phase fraction; see also Saye [24] and Kromer and Bothe [14].

Implementation

`PolygonIndicator`, `initialize!`.

4.3 Height functions and contact angle

Interface curvature κ uses local height functions with configured geometric-height-function or WY fallbacks and boundary contact angles.

Deviation 4.2: Local height-function quality

The implemented LHF quality weight uses the approved smoothstep correction to the thesis's printed polynomial.

Implementation

`InterfaceField`, `interface_height`; internal `_height_quality`.

Chapter 5

Geometric VOF transport

Thesis Chapters 3 and 6 describe geometric VOF transport and one- and two-velocity mass fluxes [20]. This chapter will specify the directional Weymouth–Yue (WY) implementation [34], its carriers, CFL restrictions, and fraction handling.

5.1 Directional WY sweeps

The indicator χ is frozen during the split update, with PLIC reconstruction between sweeps.

Deviation 5.1: Split donating regions

The thesis's two-fluid scheme uses unsplit donating regions; the package uses directional WY sweeps.

Deviation 5.2: First-moment transport

Unlike the thesis's centroid-advection formulation, no centroid or first moment is advected.

Implementation

`WYAdvectionWorkspace`, `advect!`, `stable_timestep`.

5.2 Fraction bounds and CFL

The WY limit $\nu^{\dagger}$ and incoming, outgoing, and extrusion bounds will be stated alongside the implementation's fraction-floor policy.

Deviation 5.3: Fraction snapping

Clamping or snapping fractions at the configured floor is non-conservative.

Implementation

`InterfaceField`, `stable_timestep`.

5.3 Two-velocity interface carriers

The frozen and projected-liquid policies determine phase transport; their sweep coupling will be documented here.

Deviation 5.4: Projected-liquid carrier

The projected-liquid policy and its split-sweep auxiliary gas reset are not in the thesis.

Implementation

`FrozenInterfaceFluxes`, `ProjectedLiquidVelocity`, `phase_flux_diagnostics`.

Chapter 6

Momentum transport

Thesis Chapter 6 derives mass–momentum-compatible fluxes and unsplit flux interpolants [20, 21]. This chapter will map staggered transport, its m fluxes, limiters, and AMR schedules.

6.1 Staggered fluxes and dual geometry

The AMR dual graph and the transfer of phase fluxes to staggered control volumes are the subject of this section.

Implementation

`StaggeredVelocityAdvectionWorkspace`, `MomentumAdvectionWorkspace`, `convection!`.

6.2 Flux interpolants and limiters

The implemented interpolants include Lax–Wendroff and Sweby-form limiters; see Sweby [25] and Zeng [36].

Deviation 6.1: Directional momentum advection

The package alternates directional sweeps with a variable-velocity Lax–Wendroff predictor; the thesis interpolants use unsplit donating-region centroids.

Implementation

`LaxWendroff`, `MonotonizedCentral`, `VanLeer`, `advect!`.

6.3 Phase momentum transport

This section will describe phase-momentum compatibility, the CTU switch, and periodic momentum conservation.

Implementation

`PhaseFaceScalar`, `TwoPhaseVelocity`, `advect!`.

Chapter 7

Cut cells and embedded solids

Thesis Chapter 2 introduces cut-cell geometry and Chapter 4 discusses contact curvature [20]. This chapter will specify the package's 2-D clipped geometry, dual volumes, and AMR embedded operators; the thesis's ComFLOW setting is not a complete description of these operators.

7.1 Open geometry and PLIC clipping

Open volumes and face apertures distinguish fluid from the embedded solid.

Deviation 7.1: ComFLOW boundary features

The thesis's ComFLOW-specific free-surface wave-generation boundary conditions are not implemented.

Implementation

`GeometryField`, `PLICSolidGeometry`, `refresh_geometry!`.

7.2 Cut-cell operators on AMR

The aperture-weighted gradient, pressure coefficients, and no-slip solid closure require derivations for mixed-level meshes.

Implementation

`GradientOperator`, `EllipticOperator`, `enforce_geometry_no_slip!`.

Chapter 8

Surface tension and gravity

Thesis Chapters 4 and 7 discuss surface tension, the Young–Laplace jump, and the mimetic gravity model [20]. This chapter will map those ideas to the available balanced forces and curvature handling.

8.1 CSF and conservative ghost-fluid forces

The coefficient σ and curvature κ determine the pressure jump; face curvature weighting and its fallback will be specified here.

Deviation 8.1: Aperture surface tension

The thesis's aperture surface-tension model is not implemented; the supported choices are CSF and CGFM.

Deviation 8.2: Geometric-distance GFM

The thesis's nonconservative geometric-distance GFM is not a supported surface-tension choice.

Deviation 8.3: Force stage

Surface tension is added to the predictor velocity, whereas the thesis writes it inside the scaled pressure-gradient operator.

Implementation

`SurfaceTension`, `CSF`, `CGFM`, `surface_tension!`.

8.2 Gravity and well-balancedness

The gravity vector $\boldsymbol{g}$ acts through an interface-aware potential. The well-balanced relation to the thesis gravity model remains to be checked.

Implementation

`TwoPhase`, `gravity!`.

Chapter 9

Viscous terms

Thesis Chapter 7 develops two-phase viscous stresses and geometric viscosity averaging [20]. This chapter will specify the single-phase and two-phase solvers, wall closures, and AMR consistency correction.

9.1 Single-phase viscous step

The explicit half of the single-phase Crank–Nicolson step differs from the two-phase time-level choice.

Deviation 9.1: Single-phase diffusion time level

The explicit half uses u^n in the single-phase scheme but the post-transport u^* in the thesis's two-phase scheme.

Implementation

`StaggeredDiffusionOperator`, `diffusion!`, `implicit_midpoint_diffusion!`.

9.2 Two-phase viscosity and equivalent velocity

The two-phase stress, geometric viscosity mean, and common velocity increment will be related to the equivalent-velocity formulation.

Implementation

`VariableStaggeredDiffusionOperator`, `dynamic_viscosity`, `mixture_density`.

Chapter 10

Pressure projection

Thesis Chapter 7 presents one- and two-velocity pressure operators, CMDGFM, and a mimetic gradient [20]. This chapter will document the actual projection systems and boundary conditions.

10.1 One-velocity projection

The projection is non-incremental: it solves for the pressure rather than a pressure increment.

Implementation

`ProjectionWorkspace`, `project!`, `OneVelocityModel`.

10.2 CMDGFM jump interpolation

The unknown ξ is the scaled pressure-gradient jump; β controls its interpolation. Write $R = |n_t/n_f|$ for the ratio of tangential to normal interface-normal components, s for the sum of the cross-phase inference-face mass fractions, and $x = |\widetilde{\boldsymbol{\eta}} \cdot \boldsymbol{n}_f|$ for the normal alignment.

Deviation 10.1: CMDGFM beta

The code uses $\beta = (1-s)/(R-s)$ for $R > 1$ (with clamping), rather than the thesis's $3x^2 - 2x^3$.

Deviation 10.2: CMDGFM inference and unresolved rows

Inference faces use extreme mass fractions among same-axis vertex neighbours; unresolved jump rows set $\xi = 0$.

Deviation 10.3: CMDGFM linear solver

The implementation defaults to BiCGSTAB with a tree-GMG/active-jump-ILU block preconditioner, which is not the thesis's solver.

Implementation

`CMDGFMProjection`, `CMDGFMProjectionSolver`, `cmdgfm_jump_interpolant`, `cmdgfm_gradient!`.

10.3 Mimetic projection

The implemented local constraints and active-face rule will be described with their energy and momentum identities.

Deviation 10.4: Mimetic KKT system

The code solves a locally constrained (p, ξ, λ) KKT system; the thesis solves for potentials ψ_η and (p, ψ_τ).

Implementation

`MimeticProjection`, `MimeticProjectionSolver`, `mimetic_gradient!`.

10.4 Velocity and pressure boundaries

Inflow and outflow boundary handling, the pressure condition, and the impermeability requirement of WY transport will be stated separately.

Implementation

`VelocityBoundaryConditions`, `Inflow`, `Outflow`, `ZeroGradientVelocity`.

Chapter 11

Operators and linear solvers

Thesis Chapter 7 gives pressure operators on conforming meshes but not the package's tree geometric multigrid implementation [20]. This chapter will describe AMR operators, null spaces, iterative solvers, preconditioners, and residual tolerances.

11.1 Gradient and elliptic operators

The vertex-patch inverse metric, weighted elliptic operator, and compatibility conditions will be given in the thesis notation.

Implementation

`GradientOperator`, `EllipticOperator`, `PoissonOperator`, `weighted_gradient!`.

11.2 Tree geometric multigrid

Smoothing, transfer, coarse-grid solution, and the symmetry requirements for CG will be specified for the implemented V-cycle.

Implementation

`TreeGeometricMultigridPreconditioner`, `MultigridOptions`.

11.3 Krylov methods and tolerances

This section will connect solver options and the scaled projection residual to stopping criteria.

Implementation

`EllipticSolver`, `LinearSolverOptions`, `solve!`.

Chapter 12

Refinement criteria and conservative transfer

Thesis Chapter 2 describes refinement criteria but not this package's conservative block-tree transfer [20]. The chapter will map indicators and derive cell-integral, divergence, and phase-transfer properties.

12.1 Refinement criteria

The available interface, curvature, vorticity, geometry, and interpolation error criteria will be compared with the thesis's tolerances.

Deviation 12.1: Interface refinement

The package uses a target level and buffer; the thesis's interface criterion uses a phase-fraction threshold.

Deviation 12.2: Instability refinement

The thesis's instability-wavelength refinement criterion is not implemented.

Implementation

`InterfaceRefinementIndicator`, `CurvatureRefinementIndicator`, `VorticityRefinementIndicator`, `InterpolationErrorIndicator`, `GeometryRefinementIndicator`.

12.2 Cell, face, and phase transfer

Cell restriction and prolongation, face transfer, and PLIC/phase transfer will be related to their conservation requirements.

Implementation

`restrict!`, `prolong!`, `AMRWorkspace`.

12.3 Divergence preservation and regridding

The divergence-restoring correction is applied on refinement; coarsening uses area-averaged face velocities. Proofs of the associated invariants belong here.

Implementation

`MeshAdaptationOptions`, `adapt!`, `divergence!`.

Chapter 13

Stability and accuracy

The thesis describes transport CFL limits and capillary and gravity time-step bounds in Chapters 3, 4, and 7 [20]. This chapter will collect the implemented limits, their defaults, and the known order restrictions.

13.1 Advective and capillary time steps

Limits for staggered transport, WY sweeps, and capillary waves will be expressed using the minimum mesh width h and time step δt.

Implementation

`stable_timestep`, `AdvectionOptions`, `SurfaceTension`.

13.2 Gravity time-step limit

The gravity bound depends on the phase densities $\rho^{\mathrm{l}}, \rho^{\mathrm{g}}$ and $|\boldsymbol{g}|$.

Deviation 13.1: Gravity time-step factor

The implemented bound contains an extra factor $1/2$ inside the square root and is therefore $\sqrt{2}$ stricter than the thesis's gravity bound.

Implementation

`stable_timestep`, `TwoPhase`.

13.3 Splitting and spatial accuracy

Known first-order force splitting and the analysis of the single-phase complete-step order will be distinguished from observed spatial rates.

Implementation

`step!`, `NavierStokesOptions`.

Chapter 14

Verification

Thesis Chapters 3, 4, 6, and 7 report transport and flow validation studies [20]. This chapter will connect package claims to reproducible convergence, conservation, balanced-force, and AMR tests.

14.1 Convergence and reference cases

Tables in a completed reference should be generated from reproducible verification scripts and distinguish observed rates from test thresholds.

Implementation

`simulate!`, `FlowSimulation`.

14.2 Conservation and balance

The claim-to-test index will identify phase-volume, momentum, hydrostatic, energy, and divergence checks without claiming unverified properties.

Implementation

`PhaseVolume`, `LinearMomentum`, `DivergenceNorm`, `diagnostic_history`.

Appendix A

Deviations from the thesis

This list is collected automatically from the deviation boxes in the chapters; each entry links back to the corresponding implementation difference.

1.1 Dimensionality and compressibility . . . 1
4.1 Reconstruction family . . . 7
4.2 Local height-function quality . . . 7
5.1 Split donating regions . . . 9
5.2 First-moment transport . . . 9
5.3 Fraction snapping . . . 9
5.4 Projected-liquid carrier . . . 10
6.1 Directional momentum advection . . . 11
7.1 ComFLOW boundary features . . . 13
8.1 Aperture surface tension . . . 15
8.2 Geometric-distance GFM . . . 15
8.3 Force stage . . . 15
9.1 Single-phase diffusion time level . . . 17
10.1 CMDGFM beta . . . 19
10.2 CMDGFM inference and unresolved rows . . . 19
10.3 CMDGFM linear solver . . . 19
10.4 Mimetic KKT system . . . 20
12.1 Interface refinement . . . 23
12.2 Instability refinement . . . 23
13.1 Gravity time-step factor . . . 25

Appendix B

Notation

The notation follows Remmerswaal [20], with explicit additions for the block tree and nonconforming AMR faces. Only symbols used in this reference are listed here.

$\mathcal{B}, \mathcal{T}, \mathcal{L}$ Mesh block, block tree, and set of leaf blocks (AMR).

L, h_0, h Maximum refinement level, base mesh width, and minimum mesh width.

$\mathcal{H}(f), \alpha_{c,f}$ Hanging-face segment and cell–face incidence fraction (AMR).

$\mathcal{C}, \mathcal{F}$ Sets of centred control volumes and their faces.

$\vec{\mathcal{C}}, \vec{\mathcal{F}}$ Spaces of centred and face functions.

ω_f Staggered control volume associated with face f.

$\alpha^{\mathrm{l}}, \alpha^{\mathrm{g}}$ Liquid and gas volume fractions.

$\chi, \nu^{\dagger}$ Phase indicator and WY CFL limit.

$m, \delta t$ Mass flux and time-step size.

R, s, x Normal-component ratio, inference mass-fraction sum, and normal alignment in CMDGFM.

$\mathfrak{D}, \mathfrak{S}$ Discrete divergence and gradient.

u^*, u^{**}, u^{***} Velocity after transport, diffusion, and forces, respectively.

β, ξ, λ Jump-interpolation weight, scaled pressure-gradient jump, and KKT multiplier.

κ, σ Interface curvature and surface-tension coefficient.

$\rho^{\mathrm{l}}, \rho^{\mathrm{g}}, \boldsymbol{g}$ Liquid and gas densities and gravitational acceleration.

Appendix C

API index

Public names link to the matching version of the Documenter API. The implementation boxes at the ends of sections give the relevant names in context; internal implementation names are printed without links. An explicit stable anchor for each public function is planned for the API reference.

Simulation `FlowSimulation`, `step!`, `simulate!`.

Mesh and geometry `BlockTreeMesh`, `InterfaceField`, `GeometryField`.

Models and projections `TwoPhase`, `OneVelocityModel`, `TwoVelocityModel`, `CMDGFMProjection`, `MimeticProjection`.

Operators and solvers `GradientOperator`, `EllipticOperator`, `LinearSolverOptions`, `MultigridOptions`.

Bibliography

[1] A. S. Almgren, J. B. Bell, P. Colella, L. H. Howell, and M. L. Welcome. A conservative adaptive projection method for the variable density incompressible navier–stokes equations. *Journal of Computational Physics*, 142:1–46, 1998. https://doi.org/10.1006/jcph.1998.5890.

[2] L. Batteux, T. Gallouët, R. Herbin, J.-C. Latché, and P. Poullet. Convergence of the implicit MAC-discretized navier–stokes equations with variable density and viscosity on non-uniform grids. https://arxiv.org/abs/2107.10117.

[3] M. J. Berger, M. J. Aftosmis, and S. M. Murman. Analysis of slope limiters on irregular grids. In *AIAA Paper 2005-0490*, 2005. https://doi.org/10.2514/6.2005-490.

[4] M. J. Berger and P. Colella. Local adaptive mesh refinement for shock hydrodynamics. *Journal of Computational Physics*, 82:64–84, 1989. https://doi.org/10.1016/0021-9991(89)90035-1.

[5] J. Bonelle and A. Ern. Analysis of compatible discrete operator schemes for elliptic problems on polyhedral meshes. *ESAIM: Mathematical Modelling and Numerical Analysis*, 48:553–581, 2014. https://doi.org/10.1051/m2an/2013104.

[6] J. U. Brackbill, D. B. Kothe, and C. Zemach. A continuum method for modeling surface tension. *Journal of Computational Physics*, 100:335–354, 1992. https://doi.org/10.1016/0021-9991(92)90240-Y.

[7] D. L. Brown, R. Cortez, and M. L. Minion. Accurate projection methods for the incompressible navier–stokes equations. *Journal of Computational Physics*, 168:464–499, 2001. https://doi.org/10.1006/jcph.2001.6715.

[8] C. Burstedde, L. C. Wilcox, and O. Ghattas. p4est: Scalable algorithms for parallel adaptive mesh refinement on forests of octrees. *SIAM Journal on Scientific Computing*, 33:1103–1133, 2011. https://doi.org/10.1137/100791634.

[9] M. Cai, A. Nonaka, J. B. Bell, B. E. Griffith, and A. Donev. Efficient variable-coefficient finite-volume stokes solvers. *Communications in Computational Physics*, 16:1263–1297, 2014. https://doi.org/10.4208/cicp.070114.170614a.

[10] T. C. Clevenger and T. Heister. Comparison between algebraic and matrix-free geometric multigrid for a stokes problem on adaptive meshes with variable viscosity. *Numerical Linear Algebra with Applications*, 28:e2375, 2021. https://doi.org/10.1002/nla.2375.

[11] T. V. Gerya, D. A. May, and T. Duretz. An adaptive staggered grid finite difference method for modeling geodynamic stokes flows with strongly variable viscosity. *Geochemistry, Geophysics, Geosystems*, 14:1200–1225, 2013. https://doi.org/10.1002/ggge.20078.

[12] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the navier–stokes equations and a multigrid method. *Journal of Computational Physics*, 48:387–411, 1982. https://doi.org/10.1016/0021-9991(82)90058-4.

[13] R. M. J. Kramer, C. Pantano, and D. I. Pullin. Nondissipative and energy-stable high-order finite-difference interface schemes for 2-d patch refined grids. *Journal of Computational Physics*, 228:5280–5297, 2009. https://doi.org/10.1016/j.jcp.2009.04.010.

[14] J. Kromer and D. Bothe. Third-order accurate initialization of VOF volume fractions on unstructured meshes with arbitrary polyhedral cells. https://arxiv.org/abs/2111.01073.

[15] S. F. McCormick and J. W. Thomas. The fast adaptive composite grid (FAC) method for elliptic equations. *Mathematics of Computation*, 46:439–456, 1986. https://doi.org/10.1090/S0025-5718-1986-0829618-X.

[16] P. McCorquodale, P. Colella, and H. Johansen. A cartesian grid embedded boundary method for the heat equation on irregular domains. *Journal of Computational Physics*, 173:620–635, 2001. https://doi.org/10.1006/jcph.2001.6900.

[17] M. A. Olshanskii, K. M. Terekhov, and Y. V. Vassilevski. An octree-based solver for the incompressible navier–stokes equations with enhanced stability and low dissipation. *Computers & Fluids*, 84:231–246, 2013. https://doi.org/10.1016/j.compfluid.2013.04.027.

[18] S. Popinet. Gerris: A tree-based adaptive solver for the incompressible euler equations in complex geometries. *Journal of Computational Physics*, 190:572–600, 2003. https://doi.org/10.1016/S0021-9991(03)00298-5.

[19] S. Popinet. An accurate adaptive solver for surface-tension-driven interfacial flows. *Journal of Computational Physics*, 228:5838–5866, 2009. https://doi.org/10.1016/j.jcp.2009.04.042.

[20] Ronald A. Remmerswaal. *Numerical Modelling of Variability in Liquid Impacts*. PhD thesis, University of Groningen, 2023. https://doi.org/10.33612/diss.562133319.

[21] Ronald A. Remmerswaal and Arthur E. P. Veldman. Towards a sharp, structure preserving two-velocity model for two-phase flow: transport of mass and momentum, 2022. https://arxiv.org/abs/2209.14934.

[22] Y. Saad. ILUT: A dual threshold incomplete LU factorization. *Numerical Linear Algebra with Applications*, 1:387–402, 1994. https://doi.org/10.1002/nla.1680010405.

[23] Y. Saad and M. H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7:856–869, 1986. https://doi.org/10.1137/0907058.

[24] R. I. Saye. High-order quadrature methods for implicitly defined surfaces and volumes in hyperrectangles. *SIAM Journal on Scientific Computing*, 37:A993–A1019, 2015. https://doi.org/10.1137/140966290.

[25] P. K. Sweby. High resolution schemes using flux limiters for hyperbolic conservation laws. *SIAM Journal on Numerical Analysis*, 21:995–1011, 1984. https://doi.org/10.1137/0721062.

[26] O. Tatebe. The multigrid preconditioned conjugate gradient method. In *Sixth Copper Mountain Conference on Multigrid Methods*, pages 621–634, 1993. https://ntrs.nasa.gov/citations/19940017009.

[27] J. Teunissen and U. Ebert. Afivo: A framework for quadtree/octree AMR with shared-memory parallelization and geometric multigrid methods. *Computer Physics Communications*, 233:156–166, 2018. https://doi.org/10.1016/j.cpc.2018.06.018.

[28] G. Tóth and P. L. Roe. Divergence- and curl-preserving prolongation and restriction formulas. *Journal of Computational Physics*, 180:736–750, 2002. https://doi.org/10.1006/jcph.2002.7120.

[29] P. H. A. J. M. van der Plas. *Local grid refinement for free-surface flow simulations*. PhD thesis, University of Groningen, 2017. https://research.rug.nl/en/publications/local-grid-refinement-for-free-surface-flow-simulations/.

[30] H. A. van der Vorst. Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 13:631–644, 1992. https://doi.org/10.1137/0913035.

[31] A. van Hooft. The grid adaptation algorithm based on a wavelet-estimated discretization error. Basilisk documentation. https://basilisk.dalembert.upmc.fr/sandbox/Antoonvh/the_adaptive_wavelet_algorithm.

[32] P. Vaněk, J. Mandel, and M. Brezina. Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. *Computing*, 56:179–196, 1996. https://doi.org/10.1007/BF02238511.

[33] R. W. C. P. Verstappen and M. Dröge. A symmetry-preserving cartesian grid method for computing a viscous flow past a circular cylinder. *Comptes Rendus Mécanique*, 333:51–57, 2005. https://doi.org/10.1016/j.crme.2004.09.021.

[34] G. D. Weymouth and D. K.-P. Yue. Conservative volume-of-fluid method for free-surface simulations on cartesian-grids. *Journal of Computational Physics*, 229:2853–2865, 2010. https://doi.org/10.1016/j.jcp.2009.12.018.

[35] S. T. Zalesak. Fully multidimensional flux-corrected transport algorithms for fluids. *Journal of Computational Physics*, 31:335–362, 1979. https://doi.org/10.1016/0021-9991(79)90051-2.

[36] X. Zeng. A general approach to enhance slope limiters in MUSCL schemes on nonuniform rectilinear grids. *SIAM Journal on Scientific Computing*, 38:A789–A813, 2016. https://doi.org/10.1137/140970185.

[37] W. Zhang, A. Myers, K. Gott, A. Almgren, and J. Bell. AMReX: Block-structured adaptive mesh refinement for multiphysics applications. *International Journal of High Performance Computing Applications*, 35:508–526, 2021. https://doi.org/10.1177/10943420211022811.